

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

ЛЕКЦИЯ 8b

Темы занятия

События и назначение их обработчиков

Объект события

Поток событий и делегирование

Отмена действий браузера по умолчанию

Генерация событий

Избранные события

Формы и элементы управления

События

Событие (event) – сигнал о том, что что-то произошло. События используются для реакции на действия пользователя и внутреннего взаимодействия скриптов.

Список событий, доступных в DOM:

http://www.w3schools.com/jsref/dom_obj_event.asp

Обработчики событий

Событию можно назначить *обработчик* (event handler). Это функция, которая сработает, как только событие произошло.

Далее рассматриваются различные способы назначения обработчиков.

Обработчик в атрибуте

Обработчик может быть назначен прямо в разметке – в атрибуте, который называется *онсобытие*.

Пример – назначим у кнопки обработчик события `click`:

```
<!-- внимание на одинарные кавычки для строки -->  
<input value="Нажми" onclick="alert('Клик!')"  
      type="button">
```

Обработчик в свойстве DOM-элемента

Можно назначать обработчик, используя свойство DOM-элемента *onclick*.

```
<input id="elem" type="button" value="Нажми">
<script>
    var elem = document.getElementById("elem");
    elem.onclick = function () {
        alert("Клик!");
    };
</script>
```

Назначение обработчиков

В качестве обработчика может применяться имеющаяся функция. В этом случае в атрибуте **вызываем её**:

```
<script>  
function sayClick() {  
    alert("Клик!");  
}
```

```
</script>
```

```
<input id="elem" type="button" value="Нажми"  
      onclick="sayClick()">
```

Назначение обработчиков

<!-- во что фактически превращается
назначение обработчика в атрибуте -->
<input id="elem" type="button" value="Нажми">

```
<script>  
    var elem = document.getElementById("elem");  
    elem.onclick = function (event) {  
        sayClick();  
    };  
</script>
```


Значение this в обработчике

В обработчике события **this** указывает на элемент, который вызвал событие (даже если использовать в качестве обработчика метод объекта):

```
<input id="elem" type="button" value="Нажми">
<script>
    var elem = document.getElementById("elem");
    var obj = { id: "obj", method: function () { alert(this.id) } };
    // выводит "obj"
    obj.method();

    // при клике выводит "elem", а не "obj"
    elem.onclick = obj.method;
</script>
```

Создайте кнопку, которая будет скрывать себя по нажатию
()

использовать `this` в обработчике для доступа к самому элементу

```
<input type="button" onclick="this.hidden=true" value="Нажми, чтобы спрятать">
```

Назначение обработчиков

Разобранные способы назначения обработчиков имеют фундаментальный недостаток: нельзя «повесить» несколько обработчиков на одно событие.

Методы DOM-элемента `addEventListener()` и `removeEventListener()` являются ещё одним способом назначить или удалить обработчик, при этом позволяют использовать сколько угодно любых обработчиков.

Метод `addEventListener()` – синтаксис

Назначение обработчика осуществляется вызовом `addEventListener()` с тремя аргументами:

```
element.addEventListener(event, handler[, phase]);
```

- `event` – строка с именем события;
- `handler` – ссылка на функцию, которую надо поставить обработчиком;
- `phase` – необязательный булевский аргумент, «фаза» запуска обработчика (рассмотрим позже).

Метод addEventListener() – пример

```
<input id="elem" type="button" value="Нажми" />
<script>
    function handler1() {
        alert("Ещё клик");
    }

    function handler2() {
        alert("И ещё клик");
    }

    var elem = document.getElementById("elem");
    elem.onclick = function () { alert("Клик!"); };
    // назначаем ещё два обработчика
    elem.addEventListener("click", handler1);
    elem.addEventListener("click", handler2);
</script>
```

Метод `removeEventListener()`

Удаление обработчика осуществляется вызовом `removeEventListener()`:

```
element.removeEventListener(event, handler[, phase])
```

При этом необходимо передать те же аргументы, что были у `addEventListener()` (будьте внимательны с анонимными функциями).

Который из обработчиков запустится? Что будет выведено при клике после выполнения кода?

```
button.addEventListener("click", () => alert("1"));  
button.removeEventListener("click", () => alert("1"));  
button.onclick = () => alert(2);
```

Ответ: 1 и 2.

.

Первый обработчик сработает, потому что он не был удалён методом `removeEventListener`. Чтобы удалить обработчик, необходимо передать именно ту функцию, которая была назначена в качестве обработчика. Несмотря на то, что код идентичен, в `removeEventListener` передаётся новая, другая функция

Старые версии IE (IE8-)

В IE8- вместо `addEventListener()` и `removeEventListener()` используются `attachEvent()` и `detachEvent()`:

```
var elem = document.getElementById("elem");  
// проверка - мы не в старом IE?  
if (elem.addEventListener) {  
    elem.addEventListener("click", handler1);  
} else {  
    elem.attachEvent("onclick", handler1);  
}
```


Объект события

Объект события передаётся первым аргументом в обработчик события.

Браузер записывает в объект события дополнительную информацию о событии, которую может использовать обработчик (например, координаты нажатия мыши).

*) ранее в примерах мы не использовали объект события – писали обработчики без параметров.

Объект события — некоторые свойства

`type` — тип события (строка);

`target` — элемент, на котором событие возникло;

`currentTarget` — элемент, в котором в данный момент обрабатывается событие (=this в обработчике).

В зависимости от конкретного события могут быть и специфические свойства. Например, для мыши:

`clientX` и `clientY` — координаты курсора в момент клика (относительно окна).

Объект события — пример

```
<input id="elem" type="button" value="Нажми" />
```

```
<script>
```

```
    var elem = document.getElementById("elem");
```

```
    elem.onclick = function (event) {
```

```
        // вывести тип события, элемент и координаты
```

```
        alert(event.type + " на " + event.currentTarget);
```

```
        alert(event.clientX + ":" + event.clientY);
```

```
    }
```

```
</script>
```

«Всплытие» событий

При наступлении события обработчики сначала срабатывают на самом вложенном элементе, затем на его родителе, затем выше и так далее, вверх по цепочке вложенности.

*) всплывают **почти все** события, но не все. Например, событие `focus` не всплывает.

«Всплытие» событий – пример

При щелчке на Р увидим сообщения: **p**, затем **div** и **form**:

```
<form onclick="alert('form')">  
  FORM  
  <div onclick="alert('div')">  
    DIV  
    <p onclick="alert('p')">P</p>  
  </div>  
</form>
```

«Всплытие»: target и currentTarget

```
<form id="form">
  FORM
  <div>
    DIV
    <p>P</p>
  </div>
</form>
<script>
  var form = document.getElementById("form");
  form.onclick = function (event) {
    // event.target - где событие произошло
    var target = event.target;
    // event.currentTarget - где оно обработалось (= this)
    var ct = event.currentTarget;
    alert("target = " + target.tagName + ", this=" + ct.tagName);
  };
</script>
```

«Всплытие» событий

Если хотим прекратить распространение события вверх, вызываем `stopPropagation()` у объекта события:

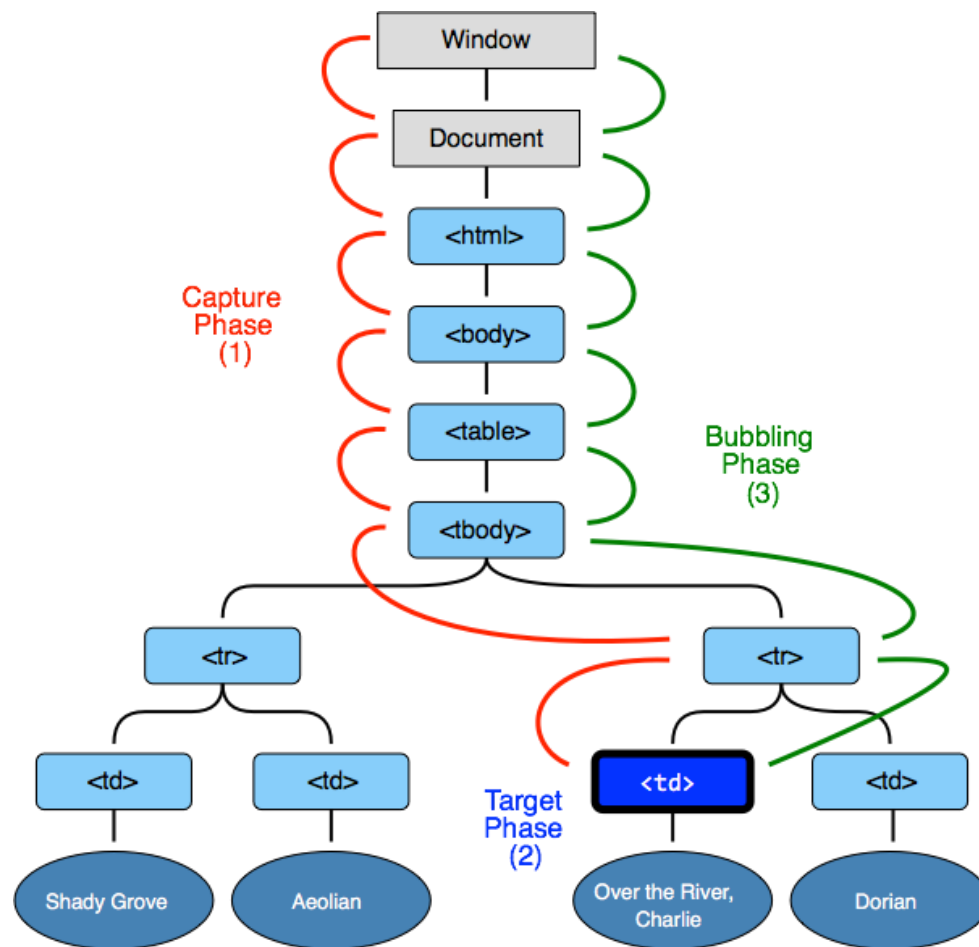
```
<body onclick="alert('сюда обработка не дойдёт')">  
  <button onclick="event.stopPropagation()">  
    Кликни  
  </button>  
</body>
```

Поток событий

Стандарт DOM2 выделяет три стадии прохода события:

1. Событие сначала идёт сверху вниз. Эта стадия называется *стадия перехвата* (capturing stage).
2. Событие достигло целевого элемента. Это – *стадия цели* (target stage).
3. После этого событие начинает всплывать. Это – *стадия всплытия* (bubbling stage).

Поток событий



Поток событий

Если хотим поймать событие на стадии «погружения», нужно добавить обработчик при помощи `addEventListener()`, где третий аргумент `true`.

Свойство `eventPhase` у объекта события показывает, на какой стадии сработал обработчик (погружение = 1, стадия цели = 2, всплытие = 3).

Поток событий – пример

```
<script>
  function handler(event) {
    alert(event.eventPhase);

    if (event.eventPhase === 3)
      event.stopPropagation();
  }

  var form = document.getElementById("form");
  form.addEventListener("click", handler, true);
  form.addEventListener("click", handler, false);

  var div = document.getElementById("div");
  div.addEventListener("click", handler, true);
  div.addEventListener("click", handler, false);
</script>
```

```
<form id="form">
  FORM
  <div id="div">
    DIV
    <p>P</p>
  </div>
</form>
```

Делегирование событий

Поток событий позволяет реализовать один из приёмов разработки – *делегирование*.

Суть приёма: если есть много элементов, события на которых нужно обрабатывать похожим образом, мы ставим один обработчик на их общего предка.

В обработчике можно получить целевой элемент (`event.target`), понять на каком именно потомке произошло событие и обработать его.

Делегирование событий – пример

Пусть имеется таблица, и необходимо обрабатывать событие `click` во всех её ячейках `td`:

```
<table id="tbl">
  <tr>
    <th colspan="3">Header</th>
  </tr>
  <tr>
    <td>TD1</td>
    <td>TD2</td>
    <td>TD3</td>
  </tr>
</table>
```

Делегирование событий – пример

```
<script>
    var table = document.getElementById("tbl");
    table.onclick = function (event) {
        // где был клик?
        var target = event.target;
        // не на TD? Тогда не интересуется
        if (target.tagName !== 'TD') return;

        target.innerHTML = "Clicked";
    };
</script>
```

Шаблон «поведение» (behavior)

Шаблон проектирования «поведение» позволяет задавать обработчики на элементе декларативно, установкой специальных HTML-атрибутов и классов.

Суть шаблона:

1. У элемента задаётся специальный атрибут, описывающий его поведение.
2. При помощи делегирования ставится обработчик на документ, который ловит все клики и, если элемент имеет нужный атрибут, производит нужное действие.

Шаблон «поведение» — пример

Сделаем так, что при клике на любом элементе с атрибутом `data-toggle-id` будет скрываться (показываться) элемент с заданным `id`.

```
<button data-toggle-id="subscribe-mail">
```

Показать форму подписки

```
</button>
```

```
<form id="subscribe-mail" hidden>
```

Ваша почта:

```
</form>
```


Шаблон «поведение» – пример

```
<script>
  document.onclick = function(event) {
    var target = event.target;
    var id = target.getAttribute("data-toggle-id");
    if (!id) return;
    var elem = document.getElementById(id);
    elem.hidden = !elem.hidden;
  };
</script>
```

Действия браузера по умолчанию

Многие события автоматически влекут за собой действие браузера. Например:

- Клик по ссылке инициирует переход на новый URL.
- Нажатие на кнопку «отправить» в форме – отсылку её на сервер.
- Двойной клик на тексте – инициирует его выделение.

Если мы обрабатываем событие в JavaScript, то часто такое действие браузера нам не нужно. К счастью, его можно отменить.

Отмена действия браузера – способ 1

Если обработчик назначен через *onсобытие* (не через `addEventListener()`), то можно просто вернуть `false` из обработчика.

В следующем примере при клике по ссылке переход не произойдёт:

```
<a href="/" onclick="return false">Нажми здесь</a>
```

Отмена действия браузера – способ 2

Можно воспользоваться объектом события. Для отмены действия браузера существует стандартный метод `event.preventDefault()`:

В следующем примере при клике по ссылке переход не произойдёт:

```
<a href="/" onclick="event.preventDefault()">Нажми</a>
```

Генерация событий

На события можно не только назначать обработчики, но и генерировать события самому.

Далее рассмотрим способ генерации событий при помощи конструктора `Event` (и подобных). Этот способ поддерживается во всех современных браузерах, за исключением IE11- (если старый браузер – смотрите тут: <https://learn.javascript.ru/dispatch-events#старое-api-для-ie9>).

Конструктор Event

Синтаксис:

```
var evt = new Event(тип_события[, флаги]);
```

тип_события – это строка, событие может быть как своим, так и встроенным, к примеру **"click"**.

флаги – объект вида {bubbles: true/false, cancelable: true/false}, где bubbles указывает, всплывает ли событие, а cancelable – можно ли отменить действие по умолчанию.

Метод `dispatchEvent()`

Этот метод запускается у элемента, чтобы инициировать событие: `elem.dispatchEvent(event)`.

При этом событие срабатывает наравне с браузерными, то есть обычные браузерные обработчики на него отреагируют. Если при создании указать `bubbles: true`, то оно будет всплывать.

Метод dispatchEvent() – пример

<!-- В этом примере эмулируем click на кнопке -->

```
<button id="btn" onclick="alert('Клик');">
```

АВТОКЛИК

```
</button>
```

```
<script>
```

```
    var btn = document.getElementById("btn");
```

```
    var event = new Event("click");
```

```
    btn.dispatchEvent(event);
```

```
</script>
```


Отмена события

На сгенерированном событии (как и на встроенном браузерном), обработчик может вызвать метод `event.preventDefault()`.

В этом случае метод `dispatchEvent()` возвратит **false**. Можно учитывать это, если мы собираемся делать какие-то действия после генерации события.

Отмена события – пример

```
var rabbit = document.getElementById("rabbit");

function hide() {
    // своё событие определили
    var event = new Event("hide", { cancelable: true });

    // генерируем и проверяем - а вдруг отмена
    if (rabbit.dispatchEvent(event))
        rabbit.hidden = true;
    else
        alert("Событие отменили");
}

// добавили обработчик
rabbit.addEventListener("hide", function (event) {
    if (confirm("Вызвать preventDefault?")) {
        event.preventDefault();
    }
});

// запустим событие через 2 секунды
setTimeout(hide, 2000);
```

```
<pre id="rabbit">
    | \      / |
      \|_ | /
        / . . \
      = \_Y_ / =
        {>o<}
</pre>
```

Справка – что есть в объекте Event?

```
interface Event {  
    // PhaseType  
    const unsigned short      CAPTURING_PHASE          = 1;  
    const unsigned short      AT_TARGET                 = 2;  
    const unsigned short      BUBBLING_PHASE           = 3;  
  
    readonly attribute DOMString     type;  
    readonly attribute EventTarget   target;  
    readonly attribute EventTarget   currentTarget;  
    readonly attribute unsigned short eventPhase;  
    readonly attribute boolean       bubbles;  
    readonly attribute boolean       cancelable;  
    readonly attribute DOMTimeStamp   timeStamp;  
  
    void stopPropagation();  
    void preventDefault();  
    void initEvent(in DOMString eventTypeArg,  
                   in boolean canBubbleArg,  
                   in boolean cancelableArg);  
  
    // All below Introduced in DOM Level 3:  
    readonly attribute DOMString     namespaceURI;  
    void stopImmediatePropagation();  
    readonly attribute boolean       defaultPrevented;  
    void initEventNS(in DOMString namespaceURIArg,  
                     in DOMString eventTypeArg,  
                     in boolean canBubbleArg,  
                     in boolean cancelableArg);  
};
```

Конструкторы других событий

Для некоторых конкретных типов событий есть свои, специфические, конструкторы. Вот некоторые:

- FocusEvent
- MouseEvent
- WheelEvent
- KeyboardEvent

Специфический конструктор позволяет указать стандартные свойства для данного типа события.

Конструкторы других событий

```
<script>
    var e = new MouseEvent("click", {
        bubbles: true,
        cancelable: true,
        clientX: 100,
        clientY: 100
    });

    alert(e.clientX); // 100
</script>
```

Конструктор CustomEvent

Этот конструктор используется для генерации собственных нестандартных событий.

Технически, он идентичен Event, кроме небольшой детали: у второго аргумента-объекта есть дополнительное свойство `detail`, в котором можно указывать информацию для передачи в событие.

CustomEvent – пример использования

```
<h1 id="h1">Привет для Васи!</h1>
```

```
<script>
```

```
  var h1 = document.getElementById("h1");
```

```
  h1.addEventListener("hello", function (event) {  
    alert(event.detail.name);  
  }, false);
```

```
  var event = new CustomEvent("hello", {  
    detail: { name: "Вася" }  
  });
```

```
  h1.dispatchEvent(event);
```

```
</script>
```

События документа

- `DOMContentLoaded` – означает, что все DOM-элементы разметки уже созданы, можно их искать, вешать обработчики, создавать интерфейс, но при этом, возможно, ещё не догрузились какие-то картинки или стили.
- `load` – страница и все ресурсы загружены, используется редко, обычно нет нужды ждать этого момента.
- `beforeunload/unload` – можно проверить, сохранил ли посетитель изменения, уточнить, действительно ли он хочет покинуть страницу.

События загрузки ресурсов

Для **script**: `onload` (после успешной загрузки и выполнения) и `onerror` (ошибка при загрузке).

Для **img** и **link**: `onload` и `onerror`.

Для **iframe**: `onload` (после загрузки)

События загрузки ресурсов – пример

```
<script>
    // динамически создаём скрипт
    var script = document.createElement("script");
    document.body.appendChild(script);

    // что делать после загрузки?
    // например, использовать функцию f() из скрипта
    script.onload = function () {
        alert(f());
    }

    // иницилируем загрузку
    script.src = "myscript.js";
</script>
```

Формы и элементы управления

Доступ к форме:

1. Традиционно – по `id`
2. По индексу в коллекции `document.forms`
3. По имени в `document.forms` (у формы должен быть задан атрибут **name**)

У элемента управления на форме определено свойство `form` – ссылка на форму, которой он принадлежит.

У формы есть коллекция `elements`.

Формы и элементы управления

```
<body>
  <form name="my">
    <input name="one" value="1">
    <input name="two" value="2">
  </form>
<script>
  // можно document.forms[0]
  var form = document.forms.my;

  // можно form.elements[0]
  var elem = form.elements.one;

  alert(elem.value); // 1
</script>
</body>
```

Формы и элементы управления

Если на форме несколько элементов с одинаковыми name, то `form.elements[name]` – это коллекция:

```
<form>
  <input type="radio" name="age" value="10">
  <input type="radio" name="age" value="20">
</form>
<script>
  var form = document.forms[0];
  var elems = form.elements.age;
  alert(elems[0].value); // 10, первый input
</script>
```

Отправка формы

При «естественной» отправке формы генерируется событие `submit`. Можно написать обработчик этого события и, при необходимости, отменить его (например, пользователь ввёл неправильные данные).

Метод `form.submit()` выполняет программную отправку формы. При этом само событие `submit` не генерируется!